

LAB ACTION PLAN FOR WEEK 5

CSE-B

24BD1A0539-P.BHARGAVI

Email-bhargavipabbathi17@gmail.com

Objectives:

Students will able to:

- Learn how to pull, run, stop, start, remove, and inspect containers and images.
- Gain the ability to create, monitor, and troubleshoot running containers.
- Configure and manage networks for container communication.
- Create and manage persistent storage for containers.
- Learn how to list, remove, and manage images efficiently.

Docker:

Docker is an open-source platform that automates the process of building, packaging, and running applications inside lightweight, portable containers, ensuring they run consistently across different environments.

Docker Images:

A read-only template that contains the application code, runtime, libraries, and dependencies. It is like a blueprint for creating containers.

Docker Containers:

A running instance of a Docker image.

```
Microsoft Windows [Version 10.0.26200.9445]
(c) Microsoft Corporation. All rights reserved.

C:\Users\Appu>docker --version
Docker version 29.1.2, build 890dcca

C:\Users\Appu>|
```

Working with Docker CLI Commands:

Docker CLI Commands with redis

Step 1: Pull the redis Image

Command:

docker pull redis

```
C:\Users\Appu>docker pull redis
Using default tag: latest
latest: Pulling from library/redis
Digest: sha256:298e5b3bc566bade82f46ad5511777a4a07a294097ce16ada2f6a42be5239df5
Status: Downloaded newer image for redis:latest
docker.io/library/redis:latest
C:\Users\Appu>
```

Step 2: Run a Redis Container

Command:

```
docker run --name my-redis -d redis
```

What It Does:

- Creates and starts a container named my-redis from the redis image.
- The -d flag runs the container in the background.

```
C:\Users\Appu>docker run --name my-redis -d redis
b223e359e212a1f0a15d27a5a4ac5ebca5aa40759e8f9ec2024b7c09ab03a3ae
C:\Users\Appu>
```

Step 3: Check Running Containers

Command:

```
docker ps
```

What It Does:

- Lists all running containers.

```
C:\Users\Appu>docker ps
CONTAINER ID   IMAGE      COMMAND                  CREATED        STATUS
PORTS         NAMES
b223e359e212  redis     "docker-entrypoint.s..." 19 seconds ago Up 16 sec
onds          6379/tcp  my-redis
C:\Users\Appu>
```

Step 4: Access Redis

Command:

```
docker exec -it my-redis redis-cli
```

Opens the Redis command-line tool (redis-cli) inside the container.

Example Redis Commands:

```
127.0.0.1:6379> SET name "Alice"
```

```
OK
```

```
127.0.0.1:6379> GET name
```

```
"Alice"
```

```
127.0.0.1:6379>exit // to come out
```

```
C:\Users\Appu>docker exec -it my-redis redis-cli
127.0.0.1:6379> SET name "Alice"
OK
127.0.0.1:6379> GET name
"Alice"
127.0.0.1:6379> |
```

Step 5: Stop the Redis Container

Command:

```
docker stop my-redis
```

What It Does:

- Stops the Redis container but doesn't delete it.

```
127.0.0.1:6379> exit

C:\Users\Appu>docker stop my-redis
my-redis

C:\Users\Appu>|
```

Step 6: Restart the Redis Container

Command:

```
docker start my-redis
```

What It Does:

- Restarts the stopped container.

```
C:\Users\Appu>docker start my-redis
my-redis

C:\Users\Appu>|
```

Step 7: Remove the Redis Container

Command:

```
docker rm my-redis // Note before removing stop the container
```

What It Does:

- Deletes the container permanently.

```
C:\Users\Appu>docker stop my-redis
my-redis

C:\Users\Appu>docker rm my-redis
my-redis

C:\Users\Appu>|
```

Step 8: Remove the Redis Image

Command:

```
docker rmi redis
```

What It Does:

- Deletes the Redis image from your local system.

```
C:\Users\Appu>docker rmi redis
Untagged: redis:latest
Deleted: sha256:298e5b3bc566bade82f46ad5511777a4a07a294097ce16ada2f6a42be523
9df5

C:\Users\Appu>|
```

2.Working with Dockerfile

A Dockerfile is a text file with instructions to create a custom Docker image.

Step 1: Set Up Your Folder

1. Windows:

- o Create a folder like C:\DockerProjects\Redis.

- o Open Git Bash and navigate to the folder:

```
cd /c/DockerProjects/Redis
```

2. Mac/Linux:

- o Create a folder:

```
mkdir ~/DockerProjects/Redis
```

```
cd ~/DockerProjects/Redis
```

Step 2: Write the Dockerfile

1. Inside the folder, create a file named Dockerfile (no extension).

2. Add the following content:

FROM redis:latest

CMD ["redis-server"]

```
C:\Users\Appu>mkdir C:\DockerProjects\Redis  
C:\Users\Appu>cd C:\DockerProjects\Redis
```

```
Appu@LAPTOP-74HF5PTB MINGW64 ~  
$ cd /c/DockerProjects/Redis  
  
Appu@LAPTOP-74HF5PTB MINGW64 /c/DockerProjects/Redis  
$ nano Dockerfile  
  
Appu@LAPTOP-74HF5PTB MINGW64 /c/DockerProjects/Redis  
$ |
```

Docker Commands (Step-by-step):

1. docker build -t redisnew.

What it does:

- This creates (builds) a Docker image using the recipe (Dockerfile) in the current folder (.).
- -t redisnew: Gives the image a name/tag ("redisnew"), so you can find it easily.

```
MINGW64:/c/DockerProjects/Redis  
$ nano Dockerfile  
  
Appu@LAPTOP-74HF5PTB MINGW64 /c/DockerProjects/Redis  
$ docker build -t redisnew .  
[+] Building 4.9s (6/6) FINISHED                                docker:desktop-linux  
=> [internal] load build definition from Dockerfile              0.1s  
=> => transferring dockerfile: 76B                               0.0s  
=> [internal] load metadata for docker.io/library/redis:latest  4.7s  
=> [auth] library/redis:pull token for registry-1.docker.io    0.0s  
=> [internal] load .dockerignore                                 0.0s  
=> => transferring context: 2B                                     0.0s  
=> CACHED [1/1] FROM docker.io/library/redis:latest@sha256:298e5b3bc566b 0.0s  
=> => resolve docker.io/library/redis:latest@sha256:298e5b3bc566bade82f4 0.0s  
=> exporting to image                                           0.1s  
=> => exporting layers                                           0.0s  
=> => exporting manifest sha256:266373b18f37ba92ce9365cca7f0ac5f70a3bbc2 0.0s  
=> => exporting config sha256:a07e7a9357aef70fcb1e6687c39518a022d16ba729 0.0s  
=> => exporting attestation manifest sha256:bbc4a3014205786e9b4d39c3d9b1 0.0s  
=> => exporting manifest list sha256:d05d8e9d6e320484cbf578d8c0102393372 0.0s  
=> => naming to docker.io/library/redisnew:latest               0.0s  
=> => unpacking to docker.io/library/redisnew:latest            0.0s  
  
Appu@LAPTOP-74HF5PTB MINGW64 /c/DockerProjects/Redis  
$ |
```

2. docker run --name myredisnew -d redisnew

What it does:

- Starts a new container (mini computer) from the redisnew image.
- --name myredisnew: Names the container "myredisnew" so it's easy to identify.

- -d: Runs the container in the background.

```
Appu@LAPTOP-74HF5PTB MINGW64 /c/DockerProjects/Redis
$ docker run --name myredisnew -d redisnew
b423e2da7f4206e0c244171aed16ae41efa3c8a19c1c1b03a633f5524e10b8d
Appu@LAPTOP-74HF5PTB MINGW64 /c/DockerProjects/Redis
$
```

3. docker ps

What it does:

- Shows a list of containers that are running right now.

```
Appu@LAPTOP-74HF5PTB MINGW64 /c/DockerProjects/Redis
$ docker ps
CONTAINER ID   IMAGE     COMMAND                  CREATED        STATUS        PORTS        NAMES
b423e2da7f4   redisnew  "docker-entrypoint.s..."  21 seconds ago Up 20 seconds  6379/tcp     myredisnew
Appu@LAPTOP-74HF5PTB MINGW64 /c/DockerProjects/Redis
$
```

4. docker stop myredisnew

What it does:

- Stops the container named "myredisnew" (like turning off a computer).

```
Appu@LAPTOP-74HF5PTB MINGW64 /c/DockerProjects/Redis
$ docker stop myredisnew
myredisnew
Appu@LAPTOP-74HF5PTB MINGW64 /c/DockerProjects/Redis
$
```

5. docker login

What it does:

- Logs you into your Docker Hub account, so you can upload images.

```
Appu@LAPTOP-74HF5PTB MINGW64 /c/DockerProjects/Redis
$ docker login
Authenticating with existing credentials... [Username: bhargavi1701]

Info → To login with a different account, run 'docker logout' followed by 'docker login'

Login Succeeded
Appu@LAPTOP-74HF5PTB MINGW64 /c/DockerProjects/Redis
$
Appu@LAPTOP-74HF5PTB MINGW64 /c/DockerProjects/Redis
$
```

6. docker ps -a

What it does:

- Shows a list of all containers, including stopped ones.

7. docker commit 0e993d2009a1 budarajumadhurika/redis1

// Note 0e993d2009a1 is container id of myredisnew

What it does:

- Takes a snapshot (saves changes) of the container with ID 0e993d2009a1 and creates a new image called budarajumadhurika/redis1.

```

Appu@LAPTOP-74HF5PTB MINGW64 /c/DockerProjects/Redis
$ docker ps -a
CONTAINER ID   IMAGE          PORTS          COMMAND                  CREATED
b423e2da7f4   redisnew       "docker-entrypoin..." About a
minute ago    Exited (0) 41 seconds ago
214642dcd0c   mywebapp       "catalina.sh run" 6 days a
go ago       Exited (143)
c99ebcc8c1c   mywebapp       "catalina.sh run" 6 days a
go ago       Created
0fa2899dbbe   mywebapp       "catalina.sh run" 6 days a
go ago       Exited (130)
393b757fd0e   mywebapp       "catalina.sh run" 6 days a
go ago       Exited (130)
fcf37f4741e   myjavapp       "/__cacert_entrypoin..." 6 days a
go ago       Exited (1)

```

8. docker images

What it does:

- Lists all images saved on your system. // will show image of budarajumadhurika/redis1

```

Appu@LAPTOP-74HF5PTB MINGW64 /c/DockerProjects/Redis
$ docker commit 8b423e2da7f4 Bhargavi1701/redis1
sha256:3d6196042b763d768890748df2aed549c72fd0fbc73d0c61d96746398333e0de

Appu@LAPTOP-74HF5PTB MINGW64 /c/DockerProjects/Redis
$ docker images
IMAGE                                ID                                DISK USAGE    CONTENT SIZE    In Use
Bhargavi1701/redis1:latest           3d6196042b76                     210MB         55.4MB         U
bhargavi1701/lms:1.0                 2e5c179558f8                     587MB         154MB         U
bhargavi1701/mydockerimg:1.0         e1cdea73d63c                     704MB         218MB         U
bhargavi1701/mywebapp:latest         c853eae4a9a5                     587MB         154MB         U
bhargavi1701/newredisimage:latest    56509a468728                     210MB         55.4MB         U
bhargavi1701/redis-test:1.0         b50e66be1171                     173MB         45MB          U
docker.elastic.co/elasticsearch/elasticsearch:8.12.0 ec72548cf833                     2.1GB         728MB         U
docker.elastic.co/kibana/kibana:8.12.0 596700740fdb                     1.75GB        424MB         U
docker.elastic.co/logstash/logstash:8.12.0 86b37d87b7ce                     1.29GB        463MB         U
gcr.io/k8s-minikube/kicbase:v0.0.50 b97074569ae9                     1.94GB        545MB         U
gcr.io/k8s-minikube/kicbase@sha256:eb4fec00e8ad70adf8e6436f195cc429825ffb85f95afcd5d8d9deb576f3e93 eb4fec00e8ad                     1.94GB        545MB         U
hello-world:latest                   5dd0d3e6e255                     25.9kB        9.49kB         U
lmscan-lmscan:latest                 c1f00e0bc2df                     9.18GB        3.13GB         U
markethub-frntendbackend-analytics-worker:latest a64cf11613c2                     316MB         72.8MB         U
markethub-frntendbackend-backend:latest c050c783d442                     254MB         58.7MB         U
mongo:6                              0a83b2f824b2                     1.05GB        270MB         U
mydockerimg:latest                   e1cdea73d63c                     704MB         218MB         U

```

9. docker push budarajumadhurika/redis1

What it does:

- Uploads the image budarajumadhurika/redis1 to Docker Hub, so others can download it.


```
Appu@LAPTOP-74HF5PTB MINGW64 /c/DockerProjects/Redis
$ docker push Bhargavi1701/redis1
Using default tag: latest
The push refers to repository [Bhargavi1701/redis1]
4f4fb700ef54: Waiting
9b30195f536d: Waiting
b1de9b9d5a81: Waiting
adba983cc2e3: Waiting
db0606565b61: Waiting
6310eb16bf42: Waiting
5b7942a2dab1: Waiting
87650244ec3e: Waiting
failed to do request: Head "https://Bhargavi1701/v2/redis1/blobs/sha256:b1de9b9d5a817a02d1ecfe32acca9b8e8b7bb361185ef2d7752768b45fa075cf": dialing Bhargavi1701:443 container via direct connection because Docker Desktop has no HTTPS proxy: connecting to Bhargavi1701:443: dial tcp: lookup Bhargavi1701: no such host
Appu@LAPTOP-74HF5PTB MINGW64 /c/DockerProjects/Redis
$
```

10. docker rm 0e993d2009a1 // container id of myredisnew

What it does:

- Deletes the container with ID 0e993d2009a1 of myredisnew.

```
Appu@LAPTOP-74HF5PTB MINGW64 /c/DockerProjects/Redis
$ docker rm 8b423e2da7f4
8b423e2da7f4
Appu@LAPTOP-74HF5PTB MINGW64 /c/DockerProjects/Redis
$
```

11. docker rmi budarajumadhurika/redis1

What it does:

- Deletes the image budarajumadhurika/redis1 from your system.

```
Appu@LAPTOP-74HF5PTB MINGW64 /c/DockerProjects/Redis
$ docker rmi Bhargavi1701/redis1
Untagged: Bhargavi1701/redis1:latest
Deleted: sha256:3d6196042b763d768890748df2aed549c72fd0fbe73d0c61d96746398333e0de
Appu@LAPTOP-74HF5PTB MINGW64 /c/DockerProjects/Redis
$ |
```

12. docker ps -a

What it does:

Shows all containers again to confirm changes.


```

MINGW64/c/DockerProjects/Redis
Appu@LAPTOP-74HF5PTB MINGW64 /c/DockerProjects/Redis
$ docker rmi Bhargavi1701/redis1
Untagged: Bhargavi1701/redis1:latest
Deleted: sha256:3d6196042b763d768890748df2aed549c72fd0fbe73d0c61d96746398333e0de

Appu@LAPTOP-74HF5PTB MINGW64 /c/DockerProjects/Redis
$ docker ps -a
CONTAINER ID   IMAGE          COMMAND                  CREATED
STATUS        PORTS
2214642dcd0c   mywebapp      "catalina.sh run"       6 days a
go      Exited (143) 6 days ago
ec99ebcc8c1c   mywebapp      "catalina.sh run"       6 days a
go      Created
90fa2899dbbe   mywebapp      "catalina.sh run"       6 days a
go      Exited (130) 6 days ago
8393b757fd0e   mywebapp      "catalina.sh run"       6 days a
go      Exited (130) 6 days ago
5fcf37f4741e   myjavapp      "/__cacert_entrypoin..." 6 days a
go      Exited (1) 6 days ago
22ddc2abae16   myjavapp      "/__cacert_entrypoin..." 6 days a
go      Created
f17c18b85d2e   ubuntu        "/bin/bash"             6 days a
go      Exited (255) 6 days ago
dfcfd9c95ec3   reids         "docker-entrypoint.s..." 6 days a
go      Exited (255) 6 days ago      0.0.0.0:6379->6379/tcp
a06c62c06805   mylms         "catalina.sh run"       6 days a
go      Exited (255) 6 days ago      0.0.0.0:8081->8080/tcp
dd5ce1149b80   afd4cf08ceb   "catalina.sh run"       6 days a
go      Exited (255) 6 days ago      0.0.0.0:8081->8080/tcp
dd97002ed889   bhargavi1701/newredisimage "docker-entrypoint.s..." 2 weeks
ago      Exited (0) 2 weeks ago
bccf04285f50   redisimage    "docker-entrypoint.s..." 2 weeks
ago      Exited (0) 2 weeks ago

```

13. docker logout

What it does:

- Logs you out of Docker Hub.

14. docker pull budarajumadhurika/redis1

What it does:

- Downloads the image budarajumadhurika/redis1 from Docker Hub.

```

Appu@LAPTOP-74HF5PTB MINGW64 /c/DockerProjects/Redis
$ docker rmi bhargavi1701/redis1
Untagged: bhargavi1701/redis1:latest
Deleted: sha256:9bf66b692e99370f82f005a29b6d877aee22776d0eeeb28f9bd4979672ddfbfb

Appu@LAPTOP-74HF5PTB MINGW64 /c/DockerProjects/Redis
$ docker pull bhargavi1701/redis1
Using default tag: latest
latest: Pulling from bhargavi1701/redis1
c21bafd6cf0f: Pull complete
Digest: sha256:9bf66b692e99370f82f005a29b6d877aee22776d0eeeb28f9bd4979672ddfbfb
Status: Downloaded newer image for bhargavi1701/redis1:latest
docker.io/bhargavi1701/redis1:latest

Appu@LAPTOP-74HF5PTB MINGW64 /c/DockerProjects/Redis
$

```

15. docker run --name myredis -d budarajumadhurika/redis1

What it does:

- Starts a new container using the image budarajumadhurika/redis1.

```
Appu@LAPTOP-74HF5PTB MINGW64 /c/DockerProjects/Redis
$ docker run --name myredis -d bhargavi1701/redis1
59e9444dfc2c3423fc2d753a1da2f821e65b649e1829c65351ec6847d854c4fa
Appu@LAPTOP-74HF5PTB MINGW64 /c/DockerProjects/Redis
$ |
```

16. docker exec -it myredisredis-cli

What it does:

- Opens the Redis command-line interface (like a terminal) inside the running container myredis.

17. SET name "Abcdef"

What it does:

- Saves a key-value pair in Redis (key = name, value = Abcdef).

18. GET name

What it does:

- Retrieves the value of the key name from Redis (it will return "Abcdef").

19. exit

What it does:

- Exits the Redis CLI.

```
Appu@LAPTOP-74HF5PTB MINGW64 /c/DockerProjects/Redis
$ docker run --name myredis -d bhargavi1701/redis1
59e9444dfc2c3423fc2d753a1da2f821e65b649e1829c65351ec6847d854c4fa
Appu@LAPTOP-74HF5PTB MINGW64 /c/DockerProjects/Redis
$ docker exec -it myredis redis-cli
127.0.0.1:6379> SET name "Abcdef"
OK
127.0.0.1:6379> GET name
"Abcdef"
127.0.0.1:6379> exit
Appu@LAPTOP-74HF5PTB MINGW64 /c/DockerProjects/Redis
$ |
```

20. docker ps -a

What it does:

- Shows all containers again to check their status.

```

MINGW64:/c/DockerProjects/Redis
127.0.0.1:6379> GET name
"Abcdef"
127.0.0.1:6379> exit

Appu@LAPTOP-74HF5PTB MINGW64 /c/DockerProjects/Redis
$ docker ps -a
CONTAINER ID   IMAGE                                COMMAND                  CREATED
STATUS        PORTS
59e9444dfc2c   bhargavi1701/redis1                "docker-entrypoint.s..." 57 seconds
Up 57 seconds  6379/tcp
2214642dcd0c   mywebapp                            "catalina.sh run"         6 days ago
Exited (143)
ec99ebcc8c1c   mywebapp                            "catalina.sh run"         6 days ago
Created
90fa2899dbbe   mywebapp                            "catalina.sh run"         6 days ago
Exited (130)
8393b757fd0e   mywebapp                            "catalina.sh run"         6 days ago
Exited (130)
5fcf37f4741e   myjavapp                            "/__cacert_entrypoin..." 6 days ago
Exited (1)
22ddc2abae16   myjavapp                            "/__cacert_entrypoin..." 6 days ago
Created
f17c18b85d2e   ubuntu                             "/bin/bash"               6 days ago
Exited (255)
dfcfdcf95ec3   reids                               "docker-entrypoint.s..." 6 days ago
Exited (255)  0.0.0.0:6379->6379/tcp
a06c62c06805   mylms                              "catalina.sh run"         6 days ago
Exited (255)  0.0.0.0:8081->8080/tcp
dd5ce1149b80   afd4cf08ceb                         "catalina.sh run"         6 days ago
Exited (255)  0.0.0.0:8081->8080/tcp

```

21. docker stop myredis

What it does:

- Stops the container myredis.

```

Appu@LAPTOP-74HF5PTB MINGW64 /c/DockerProjects/Redis
$ docker stop myredis
myredis

```

22. docker rm 50a6e4a9c326

What it does:

- Deletes the container with ID 50a6e4a9c326.

```

Appu@LAPTOP-74HF5PTB MINGW64 /c/DockerProjects/Redis
$ docker rm 59e9444dfc2c
59e9444dfc2c

Appu@LAPTOP-74HF5PTB MINGW64 /c/DockerProjects/Redis
$

```

23. docker images

What it does:

- Lists all images again to confirm which ones remain.

```

MINGW64:/c/DockerProjects/Redis
Appu@LAPTOP-74HF5PTB MINGW64 /c/DockerProjects/Redis
$ docker rm 59e9444dfc2c
59e9444dfc2c

Appu@LAPTOP-74HF5PTB MINGW64 /c/DockerProjects/Redis
$ docker images

IMAGE                                ID                                DISK  USAGE  Info + U In Use EXTRA
bhargavi1701/lms:1.0                 2e5c179558f8                     587MB 154MB  U
bhargavi1701/mydockerimg:1.0         e1cdea73d63c                     704MB 218MB  U
bhargavi1701/mywebapp:latest         c853eae4a9a5                     587MB 154MB  U
bhargavi1701/newredisimage:latest   56509a468728                     210MB 55.4MB U
bhargavi1701/redis-test:1.0         b50e66be1171                     173MB 45MB   U
bhargavi1701/redis1:latest           5b71195ab0e7                     210MB 55.4MB U
docker.elastic.co/elasticsearch/elasticsearch:8.12.0 ec72548cf833                     2.1GB 728MB  U
docker.elastic.co/kibana/kibana:8.12.0 596700740fdb                     1.75GB 424MB  U
docker.elastic.co/logstash/logstash:8.12.0 86b37d87b7ce                     1.29GB 463MB  U
gcr.io/k8s-minikube/kicbase:v0.0.50 b97074569ae9                     1.94GB 545MB  U
gcr.io/k8s-minikube/kicbase@sha256:eb4fec00e8ad70adf8e6436f195cc429825ffb85f95afcdb5d8d9deb576f3e93 eb4fec00e8ad                     1.94GB 545MB  U
hello-world:latest                   5dd0d3e6e255                     25.9kB 9.49kB U
l1mscan-l1mscan:latest               c1f00e0bc2df                     9.18GB 3.13GB U
markethub-frntendbackend-analytics-worker:latest a64cf11613c2                     316MB 72.8MB U

```

24. docker rm budarajumadhurika/redis1

What it does:

- Deletes the image budarajumadhurika/redis1 again.

```

Appu@LAPTOP-74HF5PTB MINGW64 /c/DockerProjects/Redis
$ docker rmi bhargavi1701/redis1
Untagged: bhargavi1701/redis1:latest
Deleted: sha256:5b71195ab0e7b0f5f564a6c4937b41874c58df4c6c9ef9c48a524affbcece7af

Appu@LAPTOP-74HF5PTB MINGW64 /c/DockerProjects/Redis
$

```

Step 3: Remove Login Credentials (Optional)

If you no longer need to be logged in, you can log out:

docker logout

What It Does:

- Logs you out from Docker Hub and removes your stored credentials.

```

Appu@LAPTOP-74HF5PTB MINGW64 /c/DockerProjects/Redis
$ docker logout
Removing login credentials for https://index.docker.io/v1/

Appu@LAPTOP-74HF5PTB MINGW64 /c/DockerProjects/Redis
$

```

Scenario Based Questions

1. Your application is running inside Docker, but you're not sure if the container is active. Which command helps you check running containers?

Answer: docker ps

2. You notice one of your containers is consuming high CPU and want to stop it. How do you stop a running container named web_app?

Answer: docker stop web_app

3. You've written a Dockerfile in your project directory and want to build an image named myapi. What Docker CLI command do you use?

Answer: `docker build -t myapi .`

4. You want to run your web_app container and expose its internal port 5000 on host port 8080. Which command should you use?

Answer: `docker run -p 8080:5000 web_app`

5. Your container is running, but you want to go inside and debug it using a shell. What CLI command helps you enter the container's shell?

Answer: `docker exec -it <container_name> /bin/bash`

6. You've built a few test images and want to delete one called old_api. How would you do this?

Answer: `docker rmi old_api`

7. Your app crashed in the container. You need to check the logs. How do you check this?

Answer: `docker logs <container_name>`

8. You want to start a container and let it run in the background. Which command should you use?

Answer: `docker run -d <image_name>`

9. You're not sure which container is using port 3000. How do you check?

Answer: `docker ps --filter "publish=3000"`

10. You built an image but forgot to tag it. How do you tag it now?

Answer: `docker tag <image_name><new_name>:<tag>`

11. You want to export an image to a .tar file for offline transfer. How do you export it?

Answer: `docker save -o myimage.tar <image_name>`

12. You want to restart a container automatically if it crashes. What Docker CLI command do you use?

Answer: `docker run --restart=on-failure <image_name>`

13. You want to see how much CPU/RAM each container uses. What Docker CLI command do you use? How to Control or Limit RAM in Docker? How to Measure Actual Memory Usage?

Answer: `docker stats` — Limit RAM: `docker run --memory="512m" <image_name>` — Measure actual usage: `docker stats`

14. You want your container to run a shell command before starting the app. What do you include in Dockerfile?

Answer: Include `RUN <command>` in the Dockerfile.

15. You want to create a custom Nginx image with your static files. What do you do?

Answer: Put static files in the project folder, use `FROM nginx`, then `COPY . /usr/share/nginx/html/`, and build with `docker build -t mynginx ..`

Conclusion:

Through this hands-on exploration of Docker commands and Dockerfile usage, students will get a clear picture about the end-to-end container workflow—from pulling images, creating and running containers, to customizing them with Dockerfiles. By practicing key commands such as `docker build`, `docker run`, `docker ps`, and `docker exec`, students will gain practical skills in managing containerized environments. The Dockerfile exercise reinforced the importance of layered builds, efficient image creation, and environment reproducibility. Overall, these activities strengthened the understanding of containerization principles, enabling building, testing and deploying applications in isolated, consistent environments.